Cross-Agent Continuity SaaS — Architecture Blueprint

A consolidated reference for building a SaaS that lets users continue conversations across AI providers (Claude, GPT, Gemini, etc.) when they hit rate limits, want to switch models, or need different capabilities.

Table of Contents

System Overview

The architecture has eleven tiers. The top eight form the **request path** — a user message enters at the top and a streamed assistant message comes back. The bottom three are **supporting infrastructure** that every tier depends on.

Two components are the differentiators where most of your engineering value lives:

- **Context Engine (5b)** — turns a Claude conversation into something Gemini can faithfully continue.
- **Provider Adapter (7)** — absorbs the wire-format differences between providers behind a clean interface.

Everything else is assembled from off-the-shelf parts.

The Eleven Tiers

Tier 1 — Edge (Cloudflare)

Terminates TLS, absorbs DDoS, runs WAF rules, serves cached static assets, applies bot detection. Splitting edge from gateway means you can absorb attacks without your FastAPI

workers seeing the traffic.

Tier 2 — Client (Next.js + Zustand)

- Chat UI with streaming token rendering via SSE
- File upload with progress and previews
- Model picker (per-conversation default plus per-message override)
- Zustand store for conversation state, current model, loading state
- Optimistic updates with server reconciliation on stream completion
- Presence channel for multi-device sync (later phase)

Tier 3 — Gateway Trio

3a. Auth (Clerk or Supabase Auth). JWT-based. RBAC for tenant/role distinctions. Webhook handlers for user lifecycle events.

3b. Rate Limiter (Redis). Per-user token buckets enforced before any request reaches application logic. Separate buckets for: messages/minute, tokens/day, cost/day. Returns 429 with `Retry-After` headers.

3c. API Gateway (FastAPI). Request validation (Pydantic), routing to internal services, request/response logging with PII scrubbing, request ID generation for tracing.

Tier 4 — Orchestrator

The conductor of every turn. Stateless. All state lives in Postgres and Redis.

Responsibilities:

- Turn lifecycle: validate → build context → call provider → stream → persist
- Idempotency: dedupe duplicate submissions via `idempotency_key`
- Budget enforcement: check daily/per-conversation caps before context build
- Switch event handler: triggered by user action or by router detecting 429
- Resume on failure: durable workflow via Temporal so a crashed pod doesn't lose a turn

Tier 5 — Core Services Row 1

5a. Provider Router. Holds fallback chain per user (default: `claude-opus-4-7 → gpt-4o → gemini-2.5-pro`). Implements Redis-backed circuit breakers. Cost-aware routing — short queries to cheaper models. Health probes every 30 seconds.

5b. Context Engine. The product itself. Detailed in its own section below.

5c. Stream Manager. Owns SSE/WebSocket connection lifecycle. Handles disconnects, persists partial responses for resumption, normalizes per-provider stream events to canonical

`StreamEvent`.

Tier 6 — Core Services Row 2

6a. RAG / Memory. Queries Qdrant for semantically relevant past messages and document chunks. Maintains three artifacts:

- Rolling summary (narrative)
- Extracted facts (structured key-value)
- Embedded message store (vectors)

6b. File Pipeline. On upload: parse PDFs (`pdfplumber` or `unstructured`), OCR images (`tesseract`), generate vision-model descriptions (call Claude or GPT-4V), chunk long documents, embed each chunk. All async via Celery.

6c. Tool Registry. Stores function-calling schemas in canonical JSON Schema. Translates per provider. Hosts MCP server connections for users' external integrations (Notion, Gmail, etc.).

Tier 7 — Provider Adapter (LiteLLM + Custom)

LiteLLM handles the common 80%. Your custom code handles:

- Multimodal content blocks (images, files, mixed)
- Complex tool result shapes
- Streaming-event normalization
- Provider-specific role/format quirks (Anthropic's separate `system` param, Gemini's `model` role, OpenAI's tools array)

Interface (Python protocol):

```python
class ProviderAdapter(Protocol):
    async def translate_messages(self, canonical: list[Message]) -> dict: ...
    async def translate_tools(self, tools: list[Tool]) -> dict: ...
    async def count_tokens(self, messages: list[Message]) -> int: ...
    async def stream_completion(self, request: dict) -> AsyncIterator[StreamEvent]:
    async def validate(self, request: dict) -> ValidationResult: ...
```

Tier 8 — Providers

Concrete endpoints: Anthropic, OpenAI, Google Gemini, self-hosted (vLLM/Ollama). Self-hosted is in the design from day one even if unused — the abstraction forces clean separation.

Tier 9 — Async Workers (Temporal + Celery)

Temporal for multi-step durable workflows:

- The switch process itself (fetch → compress → translate → call → persist)
- File ingestion pipelines (parse → OCR → describe → chunk → embed)

Celery for one-shot tasks:

- Embed-this-message
- Summarize-conversation
- Extract-facts
- Cost-meter (parse usage from provider response, write to ClickHouse)
- Webhook handler (provider async callbacks)

Tier 10 — Data Plane

- **Postgres** — transactional source of truth (conversations, messages, files, users)
- **Redis** — hot cache, rate limiter, queue backend, circuit breaker state, pub/sub
- **Qdrant** — vector search for RAG retrieval
- **S3 / R2** — blob storage for raw uploaded files
- **ClickHouse** — analytical storage for token-usage events, cost dashboards, billing

Run Redis as two instances (cache and queue separate) to prevent backlog from evicting cache.

Tier 11 — Cross-Cutting Infrastructure

- **Telemetry** — OpenTelemetry distributed tracing, Sentry for exceptions, Grafana dashboards
- **Secrets** — Vault or AWS Secrets Manager / KMS for provider API keys, rotation automation
- **Billing** — Stripe with usage-based metering tied to ClickHouse events
- **Feature Flags** — PostHog or LaunchDarkly for gradual rollouts, A/B tests, kill switches
- **Audit Log** — append-only Postgres table for compliance (SOC2, GDPR, HIPAA)

Canonical Data Model

Provider-agnostic. The adapter translates at the edge.

```sql
```

```sql
CREATE TABLE conversations (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL,
  title TEXT,
  pinned_context JSONB DEFAULT '[]',
  rolling_summary TEXT,
  summary_through_turn INT DEFAULT 0,
  extracted_facts JSONB DEFAULT '{}',
  current_model TEXT,
  version INT DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  archived_at TIMESTAMPTZ
);

CREATE TABLE messages (
  id UUID PRIMARY KEY,
  conversation_id UUID REFERENCES conversations(id),
  turn_index INT NOT NULL,
  role TEXT NOT NULL,
  content JSONB NOT NULL,
  model_used TEXT,
  token_counts JSONB,
  cost_usd NUMERIC(10,6),
  embedding_status TEXT DEFAULT 'pending',
  idempotency_key TEXT UNIQUE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(conversation_id, turn_index)
);
CREATE INDEX ON messages (conversation_id, turn_index);
CREATE INDEX ON messages USING GIN (content);

CREATE TABLE files (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL,
  storage_url TEXT NOT NULL,
  mime_type TEXT,
  size_bytes BIGINT,
  extracted_text TEXT,
  description TEXT,
  chunks JSONB DEFAULT '[]',
  parse_status TEXT DEFAULT 'pending',
  created_at TIMESTAMPTZ DEFAULT NOW()
```

```
);

CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID,
  action TEXT NOT NULL,
  resource_type TEXT,
  resource_id UUID,
  metadata JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

ContentBlock Variants

Stored as items inside `messages.content`:

```json
{"type": "text", "text": "..."}
{"type": "image", "file_id": "uuid"}
{"type": "file_ref", "file_id": "uuid", "selection": "pages 3-7"}
{"type": "tool_use", "id": "...", "name": "...", "input": {}}
{"type": "tool_result", "tool_use_id": "...", "content": [], "is_error": false}
```

Non-Negotiable Schema Decisions

- `token_counts` is a JSONB map keyed by provider, computed on write. Switch decisions are O(1).
- `idempotency_key` is unique and required. Retries never duplicate turns.
- `version` on conversations enables optimistic concurrency. Every update is `UPDATE ... WHERE version = $old`, retry on conflict.

The Context Engine

The algorithm that runs on every turn.

Inputs

- `conversation_id`
- `target_provider` (and target model)
- `current_user_message`

Algorithm

```
1. Fetch all canonical messages for conversation_id from Postgres.
2. Count tokens for target provider (use cached counts; compute for new messages).
3. If total <= target_window * 0.75:
     Use full conversation (passthrough)
   Else:
     Run compression assembly (see below)
4. Resolve file references for target:
     - Vision-capable target: send image bytes
     - Non-vision target: send description text
     - Long documents: send relevant chunks only
5. Translate to target provider's wire format via adapter.
6. Validate:
     - Role order (Anthropic strict alternation)
     - Tool call/result pairing
     - Size limits
     - MIME types
7. Hand off to provider adapter -> stream manager.
```

Compression Assembly (in this exact order)

1. **Pinned context** — user-marked must-include items (always)
2. **Extracted facts** — structured key-value summary (always)
3. **Rolling summary** — narrative of older turns (if older turns dropped)
4. **RAG-retrieved chunks** — top-K vector-similar items vs current user message
5. **Verbatim recent window** — last 10-20 turns in full
6. **Current user message**

Tag each section in the system prompt so the model knows what it's reading:

```
<system>You are a helpful assistant continuing a previous conversation.</system>
<facts>user_name: Mira, project: SaaS for context switching, ...</facts>
<rolling_summary>Earlier the user and I discussed...</rolling_summary>
<retrieved_context>[Older relevant turns]</retrieved_context>
<recent_turns>[Last N turns verbatim]</recent_turns>
```

Background Workers That Feed the Engine

Three workers maintain the artifacts:

- **Summary worker** — runs after every 10 new turns, regenerates rolling summary using Haiku or GPT-4o-mini
- **Fact extractor** — runs after every turn, updates extracted_facts JSONB
- **Embedder** — runs on every message and document chunk, writes vectors to Qdrant

Identity Drift

When switching mid-conversation, the new model will respond as itself, not as the previous one. Surface this honestly in the UI ("Continued on Gemini — Claude was rate-limited"). Don't try to hide it. Set a persona-named (not model-named) system prompt if you want consistency, and refer to "previous conversation" not "previous things I said."

Provider Adapter Patterns

Per-Provider Quirks

Provider	Role for assistant	System prompt	Tool format	Streaming
Anthropic	`assistant`	Top-level `system` param	`tools` with `input_schema`	SSE with content blocks
OpenAI	`assistant`	`system` role in messages	`tools` with `parameters`	SSE with deltas
Gemini	`model`	`systemInstruction` top-level	`function_declarations`	Custom SSE

Validation Before Sending

- **Role order** — Anthropic requires strict user/assistant alternation. Insert empty assistant turn if needed.
- **Tool pairing** — every `tool_use` block must have a matching `tool_result` in the next message.
- **Size limits** — image base64 has per-provider limits; switch to file API for larger.
- **MIME types** — providers accept different image formats; convert if needed.

Stream Event Normalization

```python
class StreamEvent:
    type: Literal["text", "tool_use", "tool_use_delta", "stop", "error"]
    content: str | dict | None
    usage: dict | None  # final event includes token counts
```

Every provider emits these events; the adapter is what makes that uniform.

Circuit Breakers

Redis-backed sliding window:

- 5 failures within 30 seconds → mark provider `degraded` for 60 seconds
- Router skips degraded providers, falls through chain
- Health probes every 30 seconds reset breakers on success

Always log breaker state changes. They're invaluable during incident postmortems.

Error Categories and Handling

Provider Errors

Error	Response
429 rate limit	Exponential backoff with jitter, then fall back to next provider
529 overloaded	Circuit-break the provider, skip immediately
400 context-length-exceeded	Re-run with more aggressive compression
Content filter refusal	Surface to user with provider name; try alternate provider
401 auth failure	Operational alert, rotate keys
5xx server error	Retry with backoff up to N, then fall back
Model deprecated	Hard-fail with clear message, update model registry

Format / Translation Errors

- **Orphaned tool calls** — `tool_use` without matching `tool_result` after compression dropped the result. Fix in compression: never drop a tool result if its `tool_use` is still in context.
- **Role-order violations** — after stripping turns. Validator inserts synthetic empty turn.
- **Schema-validation failures** — tool definitions don't match target schema. Adapter sanitizes.
- **MIME-type rejections** — convert image format before sending.

Streaming Errors

- **Client disconnect mid-stream** — server-side buffer persists the response; user retrieves on reconnect.
- **Provider stream dies mid-token** — store what arrived, mark message `interrupted`, allow retry.
- **Concurrent turn submission** — queue per conversation OR explicit cancellation token.
- **Browser tab closed** — same as client disconnect; let the response finish server-side.

Context Corruption from Compression

- **Hallucinated facts in summary** — never summarize the last N turns; cite source turn numbers in summary; user-visible "View original conversation" button.
- **Lost numerical precision** — extract structured facts in parallel; keep raw numbers in `extracted_facts`.
- **Garbled chronology** — explicit timestamps in summary entries.
- **Broken references** — keep file pointer references in context even when content is summarized.

Multimodal Mismatches

- **Vision description missed details** — let users mark images as pinned to force re-attachment in original form.
- **OCR errors** — keep originals forever, allow re-OCR on demand.
- **PDF table linearization** — flag tables specifically, store as JSON when possible.
- **File storage TTL** — keep originals indefinitely until user deletes; provider file APIs have their own TTLs.

Cost / Quota Runaway

- Per-user daily ceiling enforced at gateway

- Per-conversation token cap
- Provider kill-switch via feature flag for pricing emergencies
- ClickHouse dashboards on spend per user/provider/feature
- Alert on >2x daily-average spend

Concurrency / State

- Optimistic concurrency on `conversations.version`
- Idempotency keys on submission
- Soft-delete with TTL for "deleted" conversations (GDPR cascade after grace period)
- Last-writer-wins on multi-device with reconciliation

Privacy / Compliance

- Per-user data residency preferences
- Provider opt-out from training (different mechanism per provider)
- GDPR cascade delete: Postgres + Qdrant + S3 + ClickHouse + summaries
- BYOK for enterprise — KMS-encrypted per-tenant keys
- Audit log of all data access

Provider Drift

- Pin model versions explicitly (`claude-opus-4-7` not `latest`)
- Nightly contract tests against real provider APIs
- Subscribe to provider changelogs
- Adapters fail loudly on unknown response shapes

Operational Concerns

Deployment Topology

- Frontend on Vercel or Cloudflare Pages
- Backend on Kubernetes (EKS / GKE) or simpler PaaS (Render, Railway) at low scale
- Postgres via RDS / Supabase / Neon
- Redis via Upstash or ElastiCache
- Qdrant via Qdrant Cloud or self-hosted
- S3 / R2 for blobs
- ClickHouse via ClickHouse Cloud

- Temporal via Temporal Cloud

CI / CD

- Per-PR preview environments
- Contract tests against real provider APIs (nightly, not per-commit, to avoid quota burn)
- Database migration tests on a copy of production schema
- Load tests in staging with realistic conversation sizes (500+ turns)

Testing Strategy

- Unit tests for the adapter translation logic per provider
- Integration tests for the context engine with synthetic conversation fixtures of varying sizes
- End-to-end tests for the switch flow with mocked providers
- Contract tests against real providers in nightly CI
- Chaos tests: kill a provider, kill a worker, kill Postgres for 30 seconds

Observability Must-Haves

- Distributed trace from gateway through orchestrator → context engine → adapter → provider call
- Custom span attributes: `user_id`, `conversation_id`, `target_provider`, `compression_strategy`, `token_count`
- Metrics: provider latency p50/p95/p99, switch-success rate, cost per turn, compression ratio
- Logs: scrubbed of PII, structured JSON, correlated by request ID

A Replay Tool to Build Early

Given a conversation ID and target provider, dump exactly the request that would be sent. You'll use this every time something looks weird in production.

Build Sequence

Six phases. Each one is shippable. Do not skip ahead.

Phase 1 — Foundation (weeks 1-2)

Deliverables:

- Clerk auth with FastAPI middleware

- Postgres schema with indexes and version column
- Conversation service REST endpoints
- LiteLLM wired for Anthropic only
- Next.js client with Zustand and SSE streaming

Definition of done: Logged-in user can have a multi-turn streaming chat with Claude. Page refresh restores history.

Phase 2 — Multi-provider (weeks 3-4)

Deliverables:

- ProviderAdapter protocol defined
- OpenAI and Gemini adapters via LiteLLM
- Per-provider token counting with caching
- Model picker UI
- Naive truncation (drop oldest) when over context

Definition of done: User can switch from Claude to Gemini mid-conversation on threads under 50 turns and it works.

Phase 3 — Multimodal (weeks 5-6)

Deliverables:

- File upload to S3 / R2
- Celery worker for parse + OCR + vision description
- File table with extracted_text and description columns
- Adapter logic: vision target gets bytes, text target gets description
- PDF chunking with stored chunk metadata

Definition of done: User uploads an image to a Claude conversation, switches to a text-only provider, and the new model knows what was in the image.

Phase 4 — Intelligence (weeks 7-9) — the differentiator

Deliverables:

- Embedder worker writing to Qdrant
- Rolling-summary worker (Haiku-powered, runs every 10 turns)
- Fact-extraction worker
- RAG retriever in context engine
- Six-part compression assembly wired into engine

- Test fixtures with 300+ turn conversations

Definition of done: Long conversations with images, documents, and many turns switch providers without the new model losing context.

Phase 5 — Resilience (weeks 10-11)

Deliverables:

- Provider router with per-user fallback chains
- Redis-backed circuit breakers
- Cost-meter Celery worker writing to ClickHouse
- Admin dashboard for spend by user/provider/feature
- Per-user daily cost ceilings at gateway

Definition of done: Killing Anthropic in a fire drill auto-continues conversations on Gemini. One user cannot bankrupt you.

Phase 6 — Productionalize (weeks 12-14)

Deliverables:

- OpenTelemetry instrumentation across all services
- Sentry for unhandled exceptions
- Stripe usage-based billing tied to ClickHouse events
- PostHog feature flags with prompt A/B test infrastructure
- Audit log table and access middleware
- Security review and SOC2 evidence collection started

Definition of done: Launch-ready.

Post-Launch

- Conversation forking (copy-on-write at the DB level)
- Multi-device sync with WebSocket presence
- MCP tool integration
- BYOK for enterprise
- Fine-tuned routing models
- Whatever customers ask for

Tech Stack Summary

Layer	Choice
Edge	Cloudflare
Frontend	Next.js, Zustand, TailwindCSS, shadcn/ui
Auth	Clerk or Supabase Auth
Backend framework	FastAPI (Python)
LLM SDK	LiteLLM for common 80%, custom for the rest
Database	Postgres (Supabase / Neon / RDS)
Cache & queue	Redis (Upstash / ElastiCache) — two instances
Vector DB	Qdrant
Object storage	Cloudflare R2 or AWS S3
Analytics DB	ClickHouse
Workflows	Temporal for durable; Celery for one-shot
Telemetry	OpenTelemetry + Grafana + Sentry
Secrets	Vault or AWS Secrets Manager + KMS
Billing	Stripe
Feature flags	PostHog
Deployment	Kubernetes or Render / Railway

Closing Notes

The two coral-highlighted components — Context Engine and Provider Adapter — are where the work that matters lives. Everything else is plumbing. Don't get confused about that order: the plumbing is necessary, but it's not the product.

The hardest engineering judgment call is in the context engine: how aggressively to compress, what to summarize versus retrieve, when to be honest with the user about identity drift. Build

the replay tool early so you can see exactly what each model is receiving. That tool will save you more debugging time than any other piece of infrastructure.

Build in order. Ship each phase. Don't skip ahead.